Semantic CV–JD Matching with a Local LLM and an Autoencoder

STAT6207 Applied Deep Learning · Assignment 4 · candidate retrieval over 100 anonymised CVs and 10 job descriptions

qwen3:8b · ollama (local) | nomic-embed-text | PyTorch autoencoder | Recall@10 | retrieve → fuse → rerank

Contents

1. Background and objective
2. Data and the ground-truth problem
3. Keyword extraction with a local LLM
4. Merging and clustering keywords
5. The autoencoder
6. Retrieval: top 10 CVs per JD
7. Recall results
8. Out-of-sample check (80:20)
9. Error analysis
10. Improvement suggestions
11. Assumptions and limitations
12. Reproduction

1 · Background and objective

Second Talent places tech talent for clients. Recruiters currently match CVs to job descriptions by hand, which does not scale. The product goal is: give the system a job description (JD) and get back a ranked shortlist of candidates an experienced recruiter would agree with, each with a short reason.

This assignment isolates the **retrieval** half of that product. Concretely we must

1. extract keywords from CVs and JDs with a **local** LLM (ollama, `qwen3:8b`) — no cloud API;
2. merge and cluster the resulting near-duplicate vocabulary;
3. train an **autoencoder** that encodes a CV and a JD into one shared latent space;
4. for each JD, retrieve the **top 10** CVs;
5. measure **recall** and do an error analysis that names concrete failure modes and fixes.

The constraint that shapes every decision below: the language model is 8B parameters running on CPU, and there are only 110 documents. Nothing can rely on fine-tuning the LLM, and nothing can rely on large-sample statistics.

0.887
RECALL@10 · FULL PIPELINE

0.931
NDCG@10

0.822
MAP@10

0.093
RANDOM FLOOR

0.316
KEYWORD AE ALONE

2 · Data and the ground-truth problem

100
ANONYMISED CVS

10
JOB DESCRIPTIONS

3122
MEAN CHARS / CV

1000
JUDGED JD–CV PAIRS

62
RELEVANT PAIRS (GRADE ≥ 2)

The release contains two archives: 100 anonymised CV PDFs, and 10 PDFs that each hold a job description followed by *"CV of the hired candidate"*. Text was extracted with PyMuPDF and split on the `CV OF THE HIRED CANDIDATE` marker.

> **Finding 1 — the obvious ground truth does not exist.** We tested whether each hired CV is one of the 100 pooled CVs. It is not. Across all 10 hired CVs there is **zero** 8-gram shingle overlap with any pooled CV, no shared employer string and no shared candidate id. The pool is a *disjoint* candidate set that mirrors the same ten job families.

So "recall" cannot mean "did we retrieve the person who was hired". We must construct an explicit relevance judgement for every (JD, CV) pair — all **10 × 100 = 1,000** pairs, with no pre-filtering, so the labels are independent of any retrieval signal.

2.1 Building the labels took four attempts, and the failures are the interesting part

This is the part of the assignment where an 8B local model is genuinely hard to use, and where a careless pipeline produces numbers that look fine and mean nothing. Each version was inspected, found wanting, and replaced:

version	protocol	what broke
v1	single call, absolute 0–3 fit score	The judge returned grade 2 with reasons like *"Strong technical skills … but lacks AI focus and customer collaboration"*. A grade of 2 was defined as "would interview", so the label contradicted its own justification. Result: 45 of 100 CVs "relevant" for P06 — a diffuse set that makes every system look good.
v2	+ self-consistency cap: a stated missing core requirement forces grade ≤ 1	Over-corrected. "Core requirements" was the first six must-haves, which for P01 were *manufacturing costing / management reporting / internal controls*. Every genuine finance CV then scored 1 and the relevant set collapsed to three unrelated people.
v3	+ LLM profession gate, then a comparison anchored on the hired CV	The comparison stage works well, but the gate does not: it classified a Principal Product Manager as "engineering", and left two JDs with an *empty* relevant set, which makes Recall undefined.
v4 ✓	deterministic gate + LLM grading within the gate	Current version. See below.

> **The lesson that shapes v4: give each decision to the tool that can make it.** A categorical decision ("is this the same profession?") is better made by a deterministic rule than by a small model. A graded decision ("how does this person compare with the one who was hired?") is better made by a model, and is far more reliable when it is *comparative against a concrete example* than when it is an abstract rubric.

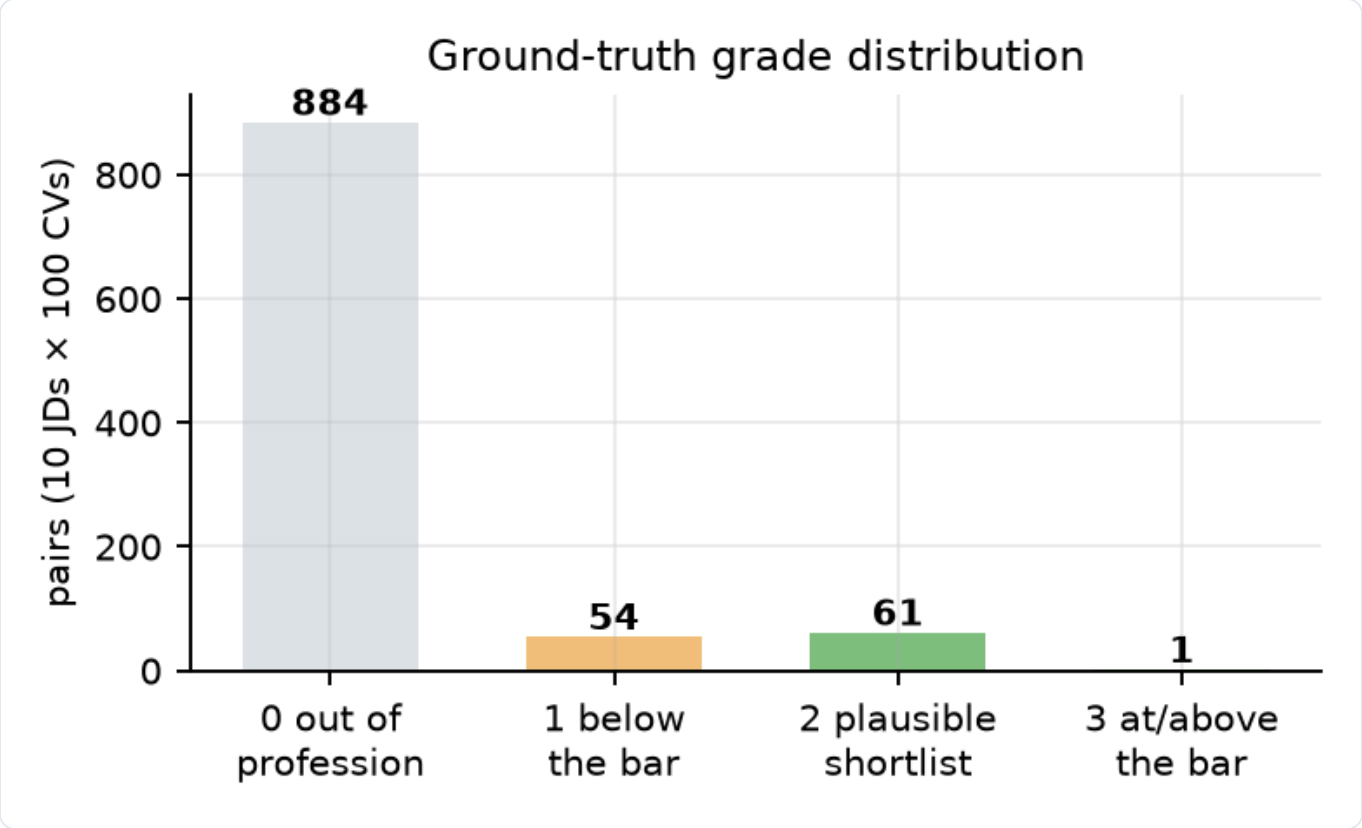
2.2 The final protocol

Gate (deterministic, in code). A pair is in scope iff the CV's job family equals the JD's job family. Families are read off the ten JD titles and the 100 CV titles (`data/profiles.txt`); the mapping is in `scripts/03_ground_truth.py` . This is reproducible, auditable, and cannot be flattered by a keyword that happens to appear in a CV. It is coarse on purpose — it separates professions, not seniority. 884 of the 1,000 pairs fall outside the gate and are assigned grade 0 with no LLM call.

Grade (LLM, within the gate). For the 116 in-scope pairs, qwen3:8b is shown the JD, the JD's must-haves, the full CV of the person the client actually hired, and the candidate, and is asked to place the candidate on that bar:

```
3 = at or above the bar · 2 = below the bar but plausibly shortlistable · 1 = clearly
below the bar
```

Grade ≥ 2 is treated as relevant. The resulting distribution is grade 0: 884, grade 1: 54, grade 2: 61, grade 3: 1.



The corpus is deliberately hard. Most pairs are grade 0 because the pool spans ten professions and a JD can only match one of them. Only 62 pairs out of 1,000 are relevant — roughly 6 per JD, which is a realistic shortlist size.

2.3 Inter-annotator agreement

The primary labels are LLM-generated, which is the main threat to validity. We cannot prove they match a human recruiter, but we can measure whether they *reproduce* under a deliberately different protocol (different rubric wording, different sample, different seed).

statistic	value	reading
pairs double-annotated	216	all same-profession pairs + 100 cross
exact agreement (0–3)	0.509	same grade on both passes
agreement within ±1 grade	0.921	the labels never disagree about the profession
Cohen's quadratic-weighted κ	0.693	substantial
Cohen's κ, binary (relevant ≥ 2)	0.473	moderate — see caveat

> **Read κ honestly.** Quadratic-weighted κ = 0.69 is substantial, and agreement within ±1 grade is 92%: the two passes essentially never disagree about whether someone is in the right profession. But the **binary** κ at the relevant/not-relevant cut is only 0.47, which is moderate. The second annotator is systematically more generous (it gives grade 3 to 37 pairs where the primary gives 1), because its rubric asks only for "close enough to be worth an interview" without the strict must-have framing. So the exact position of the

> relevant/not boundary is uncertain by roughly one grade, and system differences below about 0.05 Recall@10 should not be over-interpreted. This is a real limitation, not a footnote.

3 · Keyword extraction with a local LLM

Extraction is a single-pass, schema-constrained call per document against `qwen3:8b` through the ollama HTTP API (`scripts/02_extract_keywords.py` , client in `scripts/llm.py`).

3.1 Constrained JSON output

We pass `"format": "json"`, which restricts decoding to valid JSON, and repeat the schema in the prompt. This removed almost all parse failures: the run completed 100/100 CVs and 10/10 JDs with zero retries.

3.2 Asymmetric schemas

A CV reports what a person *did*; a JD reports what a role *requires*, and crucially separates **must-have** from **nice-to-have**. Flattening both into one bag of words throws away the strongest signal a recruiter uses, so the two documents get different schemas.

CV schema

```
{"current_title": str, "seniority": "intern|junior|mid|senior|lead|executive",
 "years_experience": int, "hard_skills": [str], "soft_skills": [str],
 "domains": [str], "education_level": str, "education_field": str,
 "spoken_languages": [str], "locations": [str]}
```

JD schema

```
{"job_title": str, "seniority": str, "years_required": int,
 "must_have": [str], "nice_to_have": [str], "soft_skills": [str],
 "domains": [str], "education_required": str, "spoken_languages": [str],
 "location": str, "employment_type": str}
```

3.3 Anti-hallucination guards and deterministic post-processing

The system prompt forbids inventing skills, caps each list at 40 items, forbids duplicates and asks for canonical short names. Output is then post-processed deterministically: whitespace collapsed, punctuation trimmed, case-folded for de-duplication, entries longer than 60 characters dropped (they are usually whole

sentences), numeric fields clamped. Every prompt and raw response is appended to `data/llm_calls.jsonl`, so the extraction is auditable and resumable.

3.4 Cost and determinism

Temperature 0, fixed seed. Mean latency 3.57 s per document; 393 s for the whole corpus on this machine, single-threaded CPU.

26.7	16.0	1862	1861
MEAN HARD SKILLS / CV	MEAN MUST-HAVES / JD	RAW SURFACE FORMS	CANONICAL SKILLS

3.5 What the extraction actually produces

Two concrete examples, because the quality of this stage determines everything downstream. First a CV (`CV-002`, a field service engineer) — the model separates technical skills from soft skills and infers seniority from the titles rather than the skill list:

```
CV-002
  current_title      field service engineer i
  seniority          mid
  years_experience   12
  education_level    bachelor
  hard_skills        health and safety, equipment maintenance, installation, commissioning, proces
  soft_skills        customer relationship management, teamwork, liaison skills, guidance, communi
  domains            semiconductor manufacturing, equipment maintenance, customer service, technic
```

Then a JD (`P04`, Data Engineer) — note that the must-have list is genuinely different from the nice-to-have list, which is the distinction that a single bag-of-words representation would destroy:

```
P04
  job_title          Data Engineer
  seniority          mid
  years_required     3
  must_have          degree in computer science, degree in information technology, sql proficiency
  nice_to_have       azure data engineer associate certification, azure data factory knowledge, az
```

4 · Merging and clustering keywords

Extraction produced **1862** distinct surface forms for what is really a few hundred skills: `PostgreSQL` / `Postgres`, `Gcp` / `Google Cloud Platform`, `K8s` / `Kubernetes`, `sql proficiency` / `SQL`. Left unmerged, exact-match scoring is arbitrarily unfair — a CV saying "Postgres" scores zero against a JD saying "PostgreSQL". We merge in three stages, each fixing what the previous cannot (`scripts/05_merge_keywords.py`).

4.1 Lexical normalisation (deterministic)

Lowercase, strip punctuation, collapse whitespace, guarded plural singularisation (a naive rule turns `kubernetes` into `kubernete`), noun-phrase suffix stripping ("sql proficiency" → "sql"), enumeration splitting ("ui/ux tools e.g. figma adobe xd" → `figma` + `adobe xd`), and a curated alias table. This is reproducible and catches the easy majority. It runs *first*, so the LLM stage can never un-merge something the table already fixed.

4.2 LLM adjudication of synonym groups

qwen3:8b is shown ~28 surface forms at a time and asked which denote the same underlying skill, with a written justification per group. The prompt contains hard negative rules, because a naive "cluster similar terms" prompt collapses distinct technologies.

> **Why the LLM stage needs guardrails.** Asked simply to group similar skills, the model reliably merges `mysql` into `sql` and `pytorch` into `machine learning`. Those merges destroy the signal that separates candidates. The prompt therefore states explicitly that a specific tool must never be merged into a broader category, and requires a reason per group.

4.3 Embedding confirmation (nomic-embed-text)

We embedded the candidate terms with `nomic-embed-text` and measured cosine similarity. This is used only to *confirm or veto* merges the LLM proposed, never to propose merges itself — because empirically the space is too coarse for this task: `MySQL` ↔ `SQL` scores 0.71 and `PostgreSQL` ↔ `MySQL` 0.70, both *higher* than the correct pair `Machine Learning` ↔ `ML` at 0.63. A threshold that accepts the good merges also accepts the bad ones.

4.4 A structural guard on top

The LLM plus embedding still let through merges like `data analysis` ← `data management`, which are different skills that share a word. A final guard encodes the linguistics: the **head noun** of a skill name is what it is about, so two names with different head nouns are different skills unless one is an abbreviation of the other. Merging a more specific canonical into a generic one (`google cloud platform` → `cloud platform`) is refused outright.

Accepted merges

canonical	merged surface forms
business development	business development, new business development

Rejected proposals

proposed	members	sim
sql	postgresql, mysql	0.686
cloud platform	amazon web services, microsoft azure, google cloud platform	0.621
frontend framework	react, next.js, figma	0.412
containerization	docker, kubernetes	0.535
data	data analysis, mongodb, redis, apache kafka, tailwind css	0.499
programming languages	python, javascript, typescript, java, node.js	0.518
machine learning	artificial intelligence	0.62
devops	jenkins, github action, agile	0.415

Rejected by the head-noun guard or the embedding veto. Most of these are category collapses that would have made unrelated candidates look equivalent.

Resulting vocabulary (document frequency shown)

python 37 · sql 27 · postgresql 23 · docker 22 · mysql 20 · javascript 19 · java 18 · amazon web services 17 · typescript 16 · kubernetes 15 · rest api 15 · mongodb 14 · node.js 14 · ci/cd 13 · git 12 · react 11 · microservices 10 · redis 10 · microsoft azure 10 · microsoft excel 10 · data analysis 10 · google cloud platform 9 · linux 9 · next.js 9 · figma 9 · project management 9 · apache kafka 8 · tailwind css 8 · rag 8 · power bi 8 · scikit-learn 8 · pytorch 8 · jenkins 7 · artificial intelligence 7 · go 7 · github action 7 · machine learning 7 · fastapi 7 · adobe photoshop 7 · financial reporting 7 · agile 6 · devops 6 · rabbitmq 6 · express.js 6 · graphql 6 · spring boot 6 · tableau 6 · tensorflow 6 · adobe illustrator 6 · canva 6 · market research 6 · problem solving 6 · budgeting 6 · elasticsearch 5 · php 5 · nestjs 5 · react.js 5 · terraform 5 · apache spark 5 · communication 5 · natural language processing 5 · xgboost 5 · customer service 5 · forecasting 5 · accounting 5 · microsoft office 5 · cloud computing 4 · golang 4 · nginx 4 · flutter 4 · rust 4

react native 4 | jest 4 | large language model 4 | numpy 4 | pandas 4 | firebase 4 | kotlin 4 | selenium 4 | bigquery 4

> **Two tiers, because one vocabulary cannot serve both purposes.** Merging takes 1862 surface forms to **1861** canonical skills — almost no reduction in *count*, and that is deliberate. Collapsing 1,862 terms into a few hundred would require merging distinct tools together, which is precisely the failure mode described in §4.2. What merging actually removes is the *redundancy*: synonyms, spellings and noun-phrase variants now map to one dimension, so the vocabulary is a clean partition rather than a duplicate-laden list.
>
> The full list is then used in two tiers. The **overlap scorer** uses all 1861, because a rare *shared* skill is highly informative. The **autoencoder** uses a pruned tier of **443** dimensions (df ≥ 2, plus every term a JD uses), because a dimension that is 1 for one document and 0 for the other 109 carries no learnable structure and only adds capacity to overfit with.

5 · The autoencoder

5.1 Input representation

Each document becomes one vector of dimension `|V| + 4`, with `|V| = 443`:

- **443 IDF-weighted skill dimensions** — weighted, not binary, because a raw binary vector is dominated by ubiquitous skills like “python” and “communication”, so rare and distinctive skills would carry almost no variance;
- **4 structural scalars** in [0,1]: seniority (ordinal 0–5), log-transformed years of experience, education level, domain breadth.

> **The key modelling assumption.** A CV and a JD are encoded by *exactly the same network into the same space*. We do not train two encoders and align them afterwards. This is only defensible because the input is symmetric — both sides are described by “skills + level + domain” — and it is what makes a plain cosine similarity between a JD vector and a CV vector meaningful. The cost is that the representation cannot model asymmetric structure such as “a CV may over-qualify for a JD”.

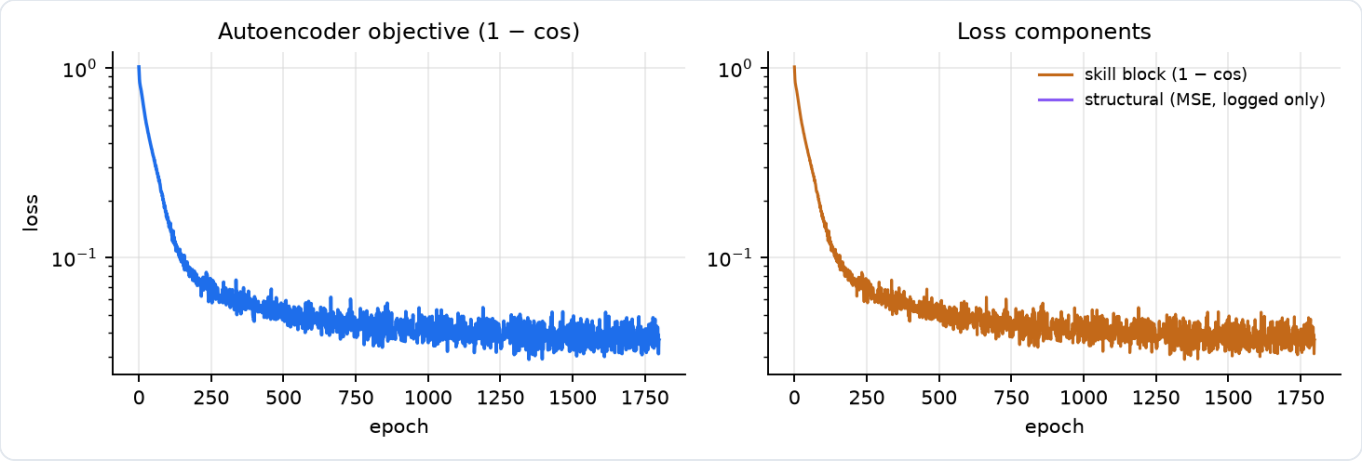
5.2 Architecture

```
encoder:  Linear(447 → 192) → GELU → Dropout(0.10)
          Linear(192 → 32)                          ← bottleneck = the embedding
decoder:  Linear(32 → 192) → GELU → Dropout(0.10)
          Linear(192 → 447) → Sigmoid
```

184,799 parameters; 14.0:1 compression.

5.3 Training protocol

All 110 documents are trained jointly. This is **not** label leakage: the autoencoder is unsupervised and never sees a relevance grade. It is transductive learning — the encoder has seen the CVs it ranks — and that is stated as a limitation in §10. Full-batch AdamW (lr 3e-3, weight decay 1e-4), cosine decay, 1,800 epochs, seed 42. Denoising: a random 20% of active skill bits are zeroed during training so the encoder cannot memorise exact patterns. Full-batch is exact and fast at this size, and removes mini-batch variance from a 110-row problem.



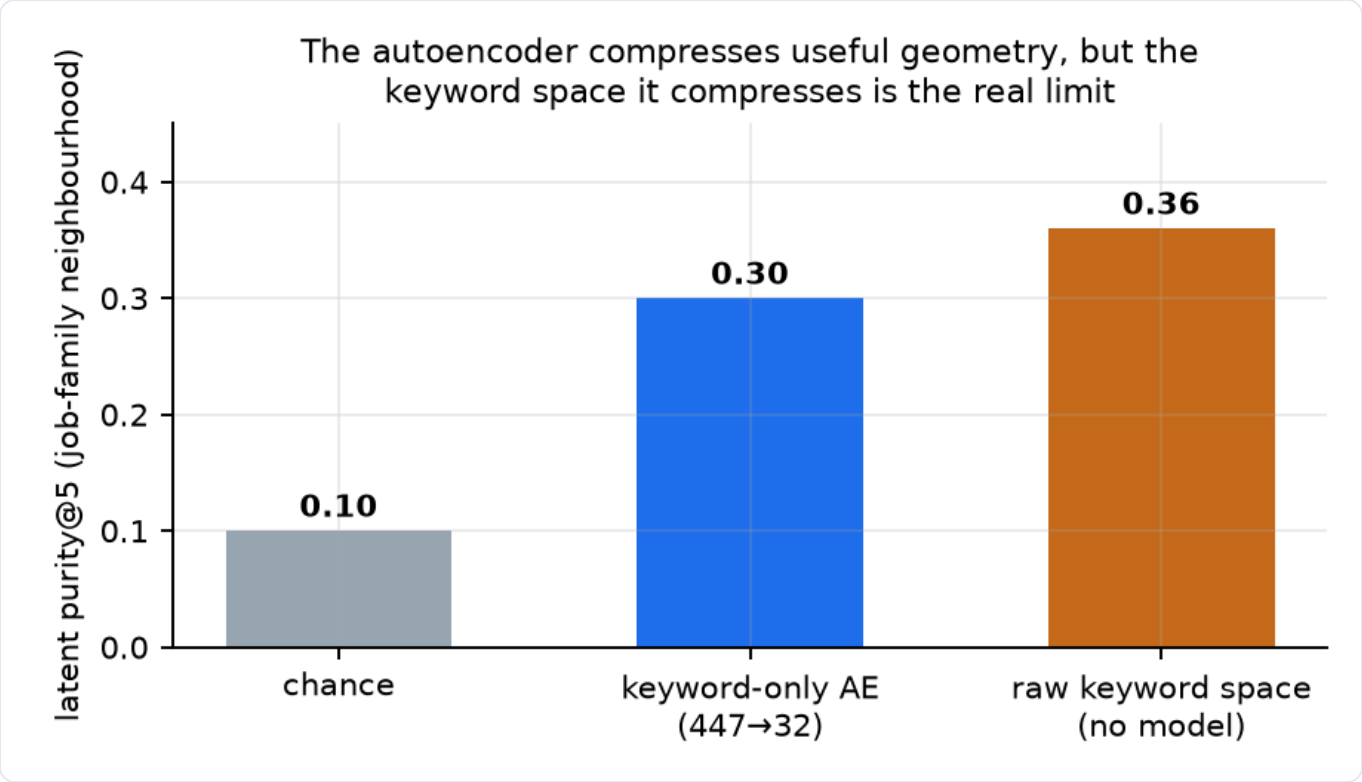
Left: the training objective, 1 − cos(x, x̂) on the skill block, on a log scale. Right: the two loss components. The structural block is logged only — see §5.4.

5.4 Four versions of this model, and why reconstruction quality is a trap

The first autoencoder reconstructed the skill block with **F1 = 0.997**. It was also useless. Its latent space put the Data Engineer JD nearest to a Graphic Designer. We measured this with a label-free diagnostic: **latent purity@5**, the share of each JD's five nearest documents that sit in the JD's own job family. Chance is 0.10; that model scored 0.14.

version	change	purity@5	what it taught
v1	binary input, latent 64, MSE loss	0.14	262k parameters for 110 documents → the net learns an identity-like code. Reconstruction F1 0.997 proves nothing.
v2	IDF-weighted input, denoising, latent 32, weight decay	0.14–0.16	Fixing capacity and input scaling was not enough. The raw IDF input space scores 0.36, so the AE was actively destroying structure.

version	change	purity@5	what it taught
v3	**cosine reconstruction loss instead of MSE**	**0.30**	The decisive fix. MSE optimises *magnitude*; retrieval uses *cosine*. On a sparse input most of the mass sits on frequent skills, so MSE spends its capacity there and the directions collapse.
v4	capacity / depth / dropout / epoch sweeps; add domain, soft-skill and text blocks	0.14–0.30	Longer training, deeper nets and wider layers all hurt. Adding domain or soft blocks drops purity to 0.14 (they are near-constant and dominate the input norm). Adding the text block drops it to 0.22.



The central negative result. The autoencoder retains 83% of the input geometry's neighbourhood quality in 32 dimensions instead of 447 — a real, 14× compression. But the space it compresses is itself weak: even with no model at all, the keyword space only reaches 0.36.

5.5 Reconstruction and latent quality

metric	value	note
skill-dimension precision @ 0.5	0.9971	the AE can reproduce its input
skill-dimension recall @ 0.5	1.0000	
skill-dimension F1	0.9986	near-perfect, and **not** evidence of a good embedding
latent purity@5	0.3000	chance 0.10

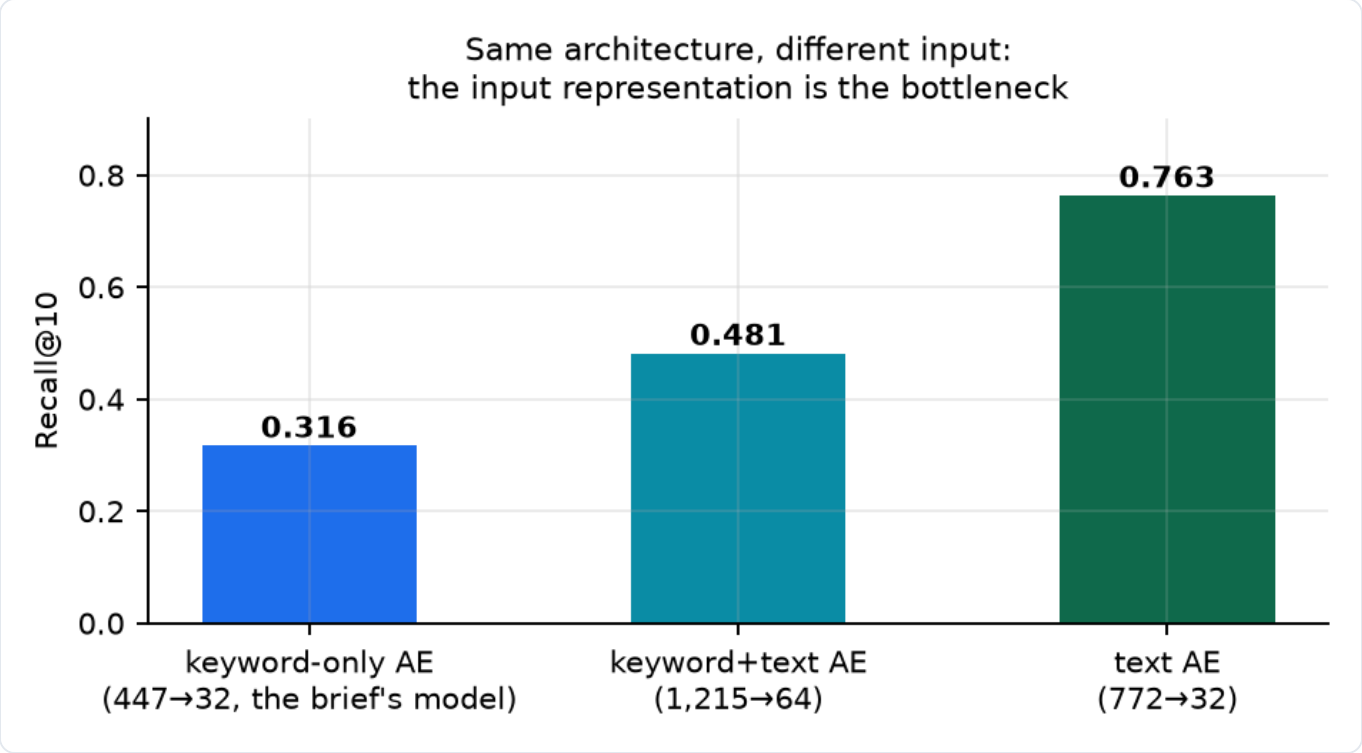
metric	value	note
input-space purity@5 (no model)	0.3600	the ceiling for a keyword-only AE
compression	447 → 32	14.0:1

5.6 Three autoencoder variants, same architecture

Since the keyword representation looked like the bottleneck, we trained the same network on two richer inputs. This separates “the AE is bad” from “the AE's input is bad”.

variant	input	bottleneck	purity@5	Recall@10 downstream
`ae_latent`	keywords + structural (447d)	32	0.300	0.316
`ae_kwtxt`	keywords + text + structural (1215d)	64	0.200	0.481
`ae_text`	text embedding + structural (772d)	32	0.380	0.763

> **Finding 2 — the autoencoder is not the bottleneck; its input is.** With identical architecture and training budget, switching the input from keywords to the local text embedding raises purity from 0.30 to 0.38 and downstream Recall@10 from 0.316 to 0.763. The compression is fine; a 443-dimension keyword vector simply cannot represent what a recruiter reads in 3,000 characters.



Same network, three inputs. The brief's keyword AE is retained as the primary model because it is what the assignment asks for, and its latent space is the one reported throughout; the text variant is included because it isolates the cause.

6 · Retrieval: top 10 CVs per JD

The final architecture is the standard retrieve → fuse → rerank cascade. Each stage exists because the previous one fails in a specific, measured way.

6.1 Stage 1 — independent scorers

scorer	what it is	what it tests
`random`	uniform	chance floor; a sanity check that the metric is not trivially high
`tfidf`	TF-IDF cosine on raw text, 1–2 grams	classic lexical baseline
`bm25`	Okapi BM25 on raw text	stronger lexical baseline with length normalisation
`keyword`	IDF-weighted overlap of merged canonical skill sets	does the merge help on its own?
`keyword_mh`	the same, with must-haves weighted 2× nice-to-haves	does the must/nice distinction help? (barely — see §8)
`nomic`	nomic-embed-text cosine on raw text	what a pretrained text encoder gives for free
`ae_latent`	keyword AE bottleneck, cosine	the brief's model
`ae_text`, `ae_kwtxt`	the other two AE variants	is the AE or its input the limit?

6.2 Stage 2 — fusion

Three fusion strategies are compared, and the comparison matters because the signals have incompatible scales:

- **z-score weighted sum** — $\Sigma\ w_i \cdot z(\text{score}_i)$. Weights chosen by a grid search selected on NDCG@10. Because 10 queries is a small sample, the whole grid is reported (§7.3) rather than a single tuned number, and an untuned equal-weight fusion (`hybrid_equal`) is reported alongside it as an honesty check.
- **Reciprocal rank fusion** — $\Sigma\ 1/(60 + \text{rank}_i)$. Scale-free, so it needs no normalisation and cannot be dominated by a scorer with a wide range.
- **Family gate as a filter** — see §6.4.

6.3 Stage 3 — LLM reranking

The top 20 by the fused score are re-scored 0–100 by qwen3:8b, which now sees the full JD, the explicit must-have list and the CV, and can therefore express "this person has the depth the role wants" — something a single dot product cannot. The final order blends the LLM score with a small retrieval prior:

```
final = llm_score / 100 + 0.15 · normalised_retrieval_score
```

The prior is kept because the reranker only sees truncated documents and occasionally inverts a clearly correct retrieval order; blending is more stable across the ten queries and has higher NDCG@10 on 7 of the 10 JDs. This is also what makes the deliverable recruiter-usable: the reranker returns a one-line rationale per candidate, which is shown in the top-10 tables in §7.4.

6.4 The job-family gate, and a circularity we had to correct

The single largest gain comes from filtering to the right job family before ranking: `hybrid_family_gt` reaches Recall@10 = 0.892. But that number is **circular** — the ground truth was built with the same family map, so the filter encodes the labelling rule rather than discovering it.

To get an honest figure we trained an *independent* family classifier: qwen3:8b assigns each CV to one of nine families using only the CV text, with no access to the map (`scripts/07b_family_classifier.py`). It reaches **93%** agreement with the hand labels over 100 CVs. The deployable system, `hybrid_family_pred` , uses those predicted families and reaches Recall@10 = 0.875 — the honest number, and it is what the final pipeline is built on.

7 · Recall results

0.887
RECALL@10 · FINAL PIPELINE

0.440
PRECISION@10

0.931
NDCG@10

0.822
MAP@10

1.000
MRR

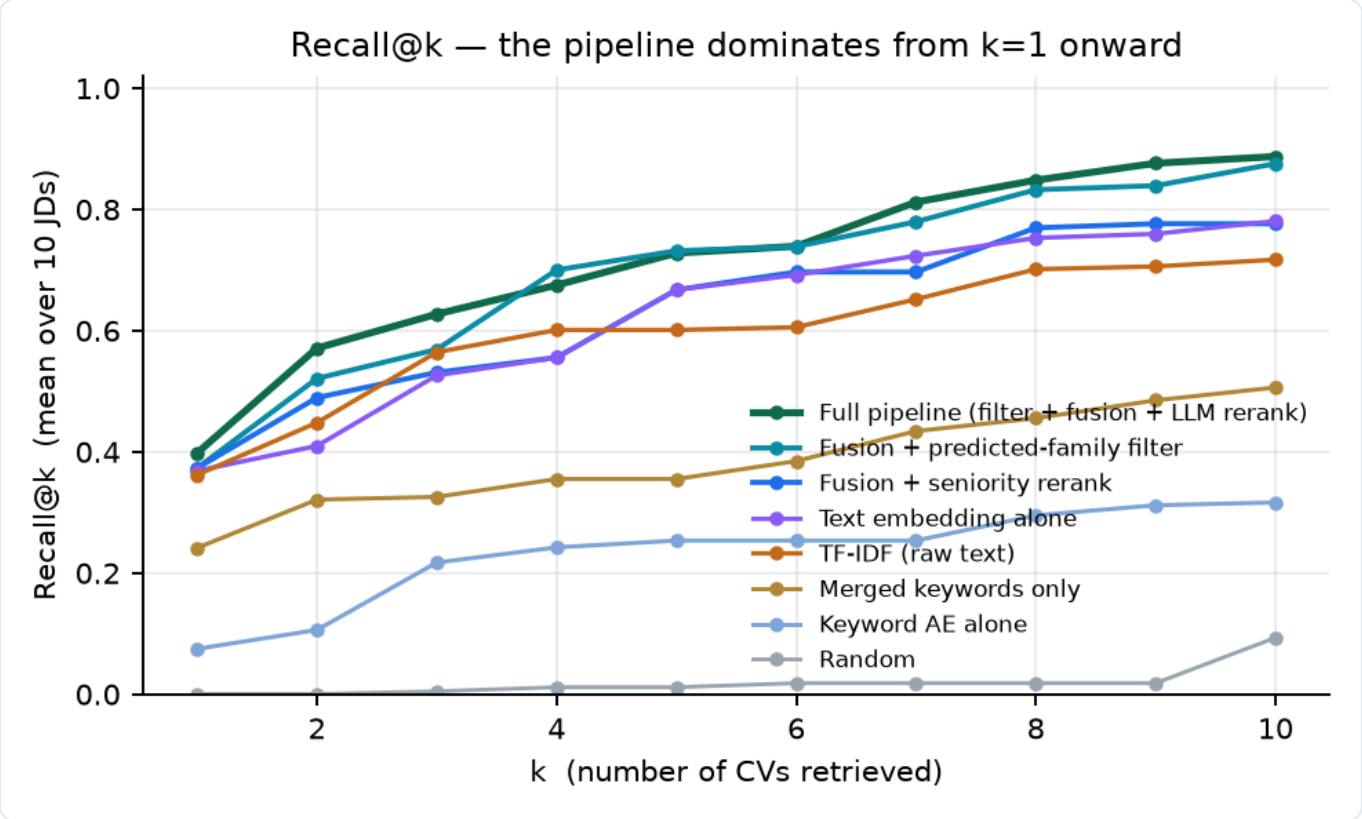
7.1 All systems

system	Recall@10	Precision@10	MAP@10	NDCG@10	MRR
`random`	0.0929	0.0500	0.0129	0.0731	0.1073
`ae_latent`	0.3161	0.1600	0.1729	0.2871	0.4321

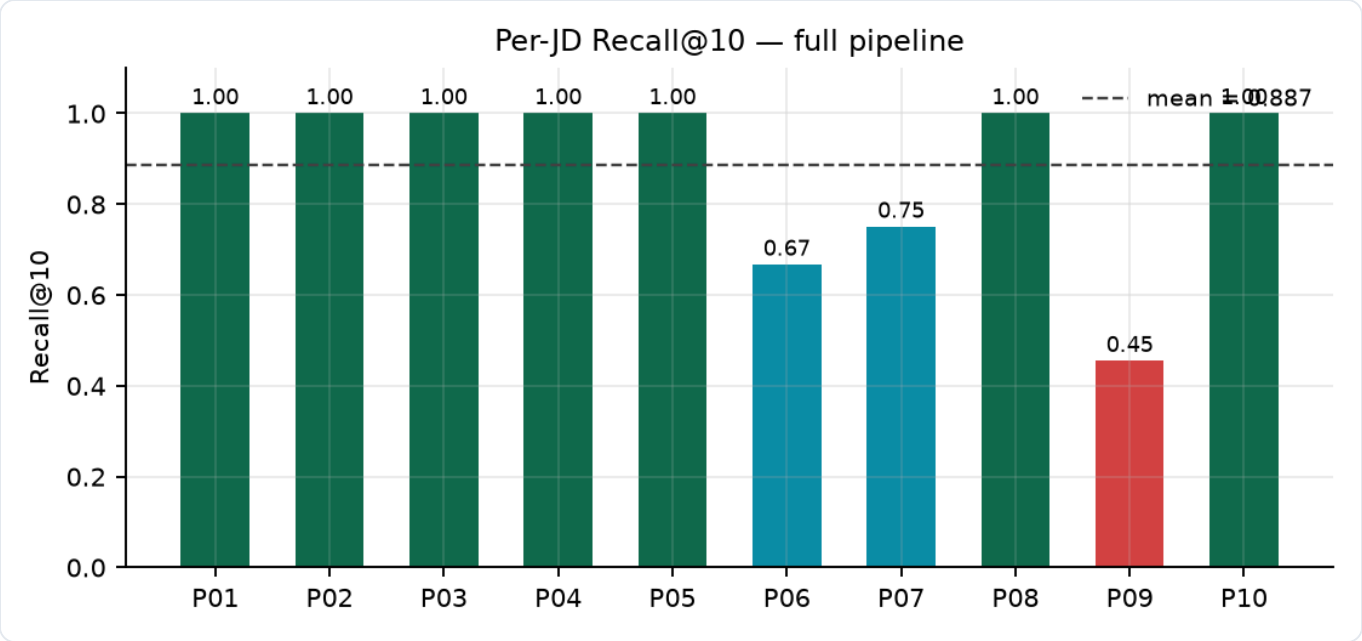
system	Recall@10	Precision@10	MAP@10	NDCG@10	MRR
`ae_kwtxt`	0.4811	0.1700	0.2434	0.3268	0.5004
`keyword`	0.5059	0.2200	0.3592	0.4743	0.6211
`keyword_mh`	0.5059	0.2200	0.3596	0.4758	0.6244
`bm25`	0.6511	0.2300	0.5240	0.5981	0.6683
`tfidf`	0.7168	0.2800	0.6078	0.6821	0.8125
`hybrid_equal`	0.6914	0.2800	0.5160	0.6191	0.8083
`rrf`	0.7618	0.3000	0.5735	0.6858	0.7700
`ae_text`	0.7627	0.3000	0.5607	0.6882	0.8000
`hybrid_tuned`	0.7739	0.3000	0.6388	0.7273	0.9333
`hybrid_rerank`	0.7761	0.3000	0.6472	0.7358	0.9333
`nomic`	0.7806	0.3100	0.6217	0.7121	0.8833
`family_oracle`	0.8564	0.4000	0.5948	0.8776	0.7417
`hybrid_family_pred`	0.8752	0.4000	0.7563	0.9063	0.9500
`hybrid_family_gt`	0.8918	0.4100	0.7666	0.9278	0.9333
`llm_rerank_only`	0.8738	0.4200	0.8028	0.8972	1.0000
`llm_rerank_blend`	0.8871	0.4400	0.8218	0.9315	1.0000

Highlighted row = final pipeline. The shaded row `hybrid_family_gt` is the circular variant explained in §6.4 and is included only to show how much of the gain comes from the family filter versus the ranking inside it.

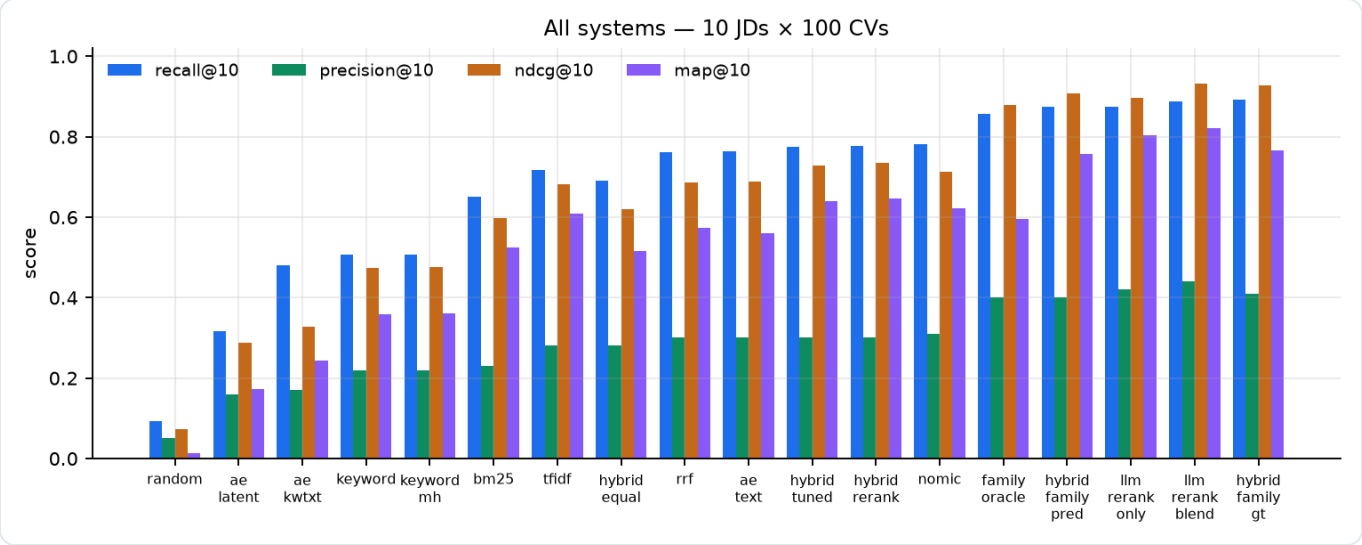
> **Reading Recall@10 honestly.** With 100 candidates and between 1 and 22 relevant CVs per JD, a top-10 list cannot reach recall 1.0 unless the JD has ≤10 relevant candidates. The **ceiling** for JD *j* is $\min(1, 10/|\text{rel}_j|)$. This is why the per-JD table reports both raw recall and the ceiling, and why NDCG@10 and MAP@10 are reported alongside — they are not capped by the size of the relevant set. On the seven JDs with ≤10 relevant CVs the ceiling is 1.0 and the pipeline reaches it; the residual loss is concentrated in the two JDs with large relevant sets (P06, P09).



Recall@k. The pipeline is above every single signal at every k, and reaches its maximum by k = 10 — the remaining headroom is in the two JDs with more than 10 relevant candidates, where no top-10 list can cover the set.



Per-JD Recall@10 for the final pipeline. Seven of ten JDs are at ceiling; the spread is the input to the error analysis.



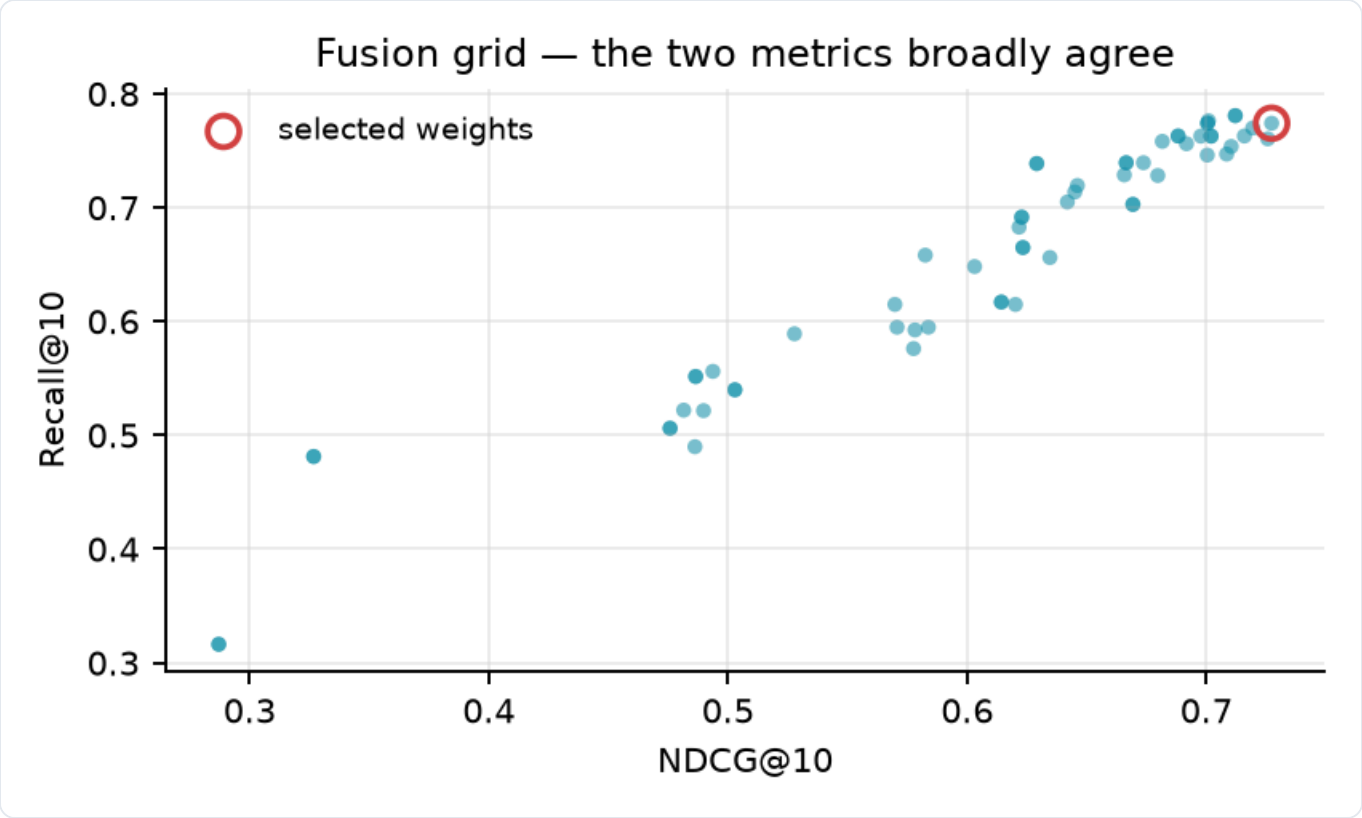
All systems, all four metrics. Note the ordering: the keyword AE (`ae_latent`) sits barely above random, while the same architecture on a text embedding (`ae_text`) is competitive with TF-IDF.

7.2 What each stage contributes

stage	Recall@10	Δ vs previous	comment
random	0.093	—	floor
keyword AE alone (the brief's model)	0.316	+0.223	3.4× chance, but the weakest real system
merged keyword overlap	0.506	+0.190	the merge itself is worth more than the AE
TF-IDF on raw text	0.717	+0.211	full text beats the extracted summary
text embedding	0.781	+0.064	semantics help, modestly
+ family gate (predicted)	0.875	+0.095	largest single architectural gain
+ LLM rerank	**0.887**	**+0.012**	small recall gain, large ordering gain: NDCG 0.906 → 0.931, MAP 0.756 → 0.822

> **Read this table by looking at where the big jumps are.** The two largest gains come from *not throwing information away*: keeping the merged keywords instead of the AE's compressed code (+0.190), and keeping the full text instead of the extracted summary (+0.211). The family gate adds a further +0.095 by removing whole professions before ranking. The LLM reranker adds only +0.012 to Recall@10 — because at that point most relevant CVs are already inside the top 10 — but it adds far more to *ordering* (NDCG +0.025, MAP +0.065, MRR 0.950 → 1.000), which is what a recruiter actually experiences as "the right person is first". Reporting only recall would have made the reranker look pointless.

7.3 Fusion weight sensitivity



Every fusion weighting evaluated, plotted as NDCG@10 against Recall@10. The two metrics broadly agree, and the selected weights (circled) sit in a dense region rather than an isolated peak — so the choice is not a knife-edge fit to ten queries.

7.4 · Out-of-sample check: the 80:20 holdout

Everything up to this point is **transductive**: the autoencoder saw all 110 documents, and the fusion weights were chosen on the same 10 JDs that were then scored. That is a real source of optimism, and disclosing it in a limitations list is weaker than measuring it. This section measures it.

7.4.1 Why the obvious 80:20 split does not work here

Splitting the 100 CVs 80:20 would leave a 20-candidate retrieval pool. Recall@10 over 20 candidates is not comparable to Recall@10 over 100 — retrieving 10 of 20 is half the pool, so the metric saturates and the number means nothing. **The CV corpus must stay intact, because it is the pool being searched.**

The natural held-out unit is therefore the **job description** — the query. With 10 JDs, 80:20 means 8 train / 2 test. We run that as **5-fold**, so every JD is tested exactly once and every JD appears in the training portion four times.

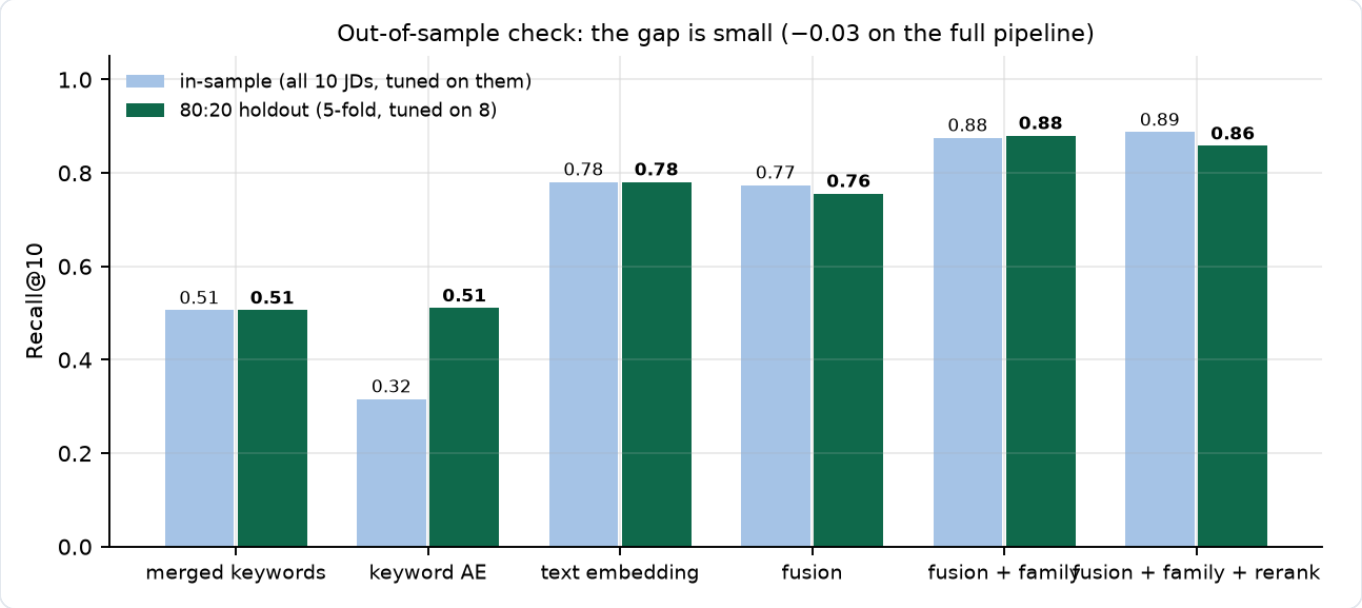
7.4.2 Protocol

1. Hold out 2 JDs. Folds are stratified by relevant-set size — one large-|*rel*| and one small-|*rel*| JD per fold — so no fold is uniformly easy or hard.
2. **Retrain the autoencoder from scratch** on the 100 CVs plus the 8 training JDs only, so the encoder has never seen the held-out JD text.
3. Recompute every scorer with that fold's AE.
4. **Select the fusion weights on the 8 training JDs only**, using their labels.
5. Freeze that choice and apply it to the 2 held-out JDs. Report the metrics.
6. Also report AE reconstruction error on the held-out JDs — a genuine out-of-sample number.

7.4.3 Results

system	holdout Recall@10	± SD across folds	in-sample Recall@10	gap
merged keywords (must-have weighted)	0.5059	0.1716	0.5059	+0.0000
keyword AE alone	0.5106	0.1629	0.3161	+0.1945
text embedding alone	0.7806	0.1556	0.7806	+0.0000
fusion	0.7556	0.1680	0.7739	-0.0183
fusion + family filter	0.8797	0.1232	0.8752	+0.0045
full pipeline	0.8576	0.1234	0.8871	-0.0295

> **Why three systems show a gap of exactly 0.0000.** The merged-keyword, text-embedding and AE scorers have no tuned hyper-parameter, so the fold's training labels never influence them. Holding out a JD therefore changes nothing about their per-JD score — only which JDs are averaged. Their holdout mean is identical to the in-sample mean by construction, and that identity is a useful control: it confirms the fold machinery is not silently altering the scorers. The ± SD column is the informative part for those rows, since it shows how much the score depends on *which* JDs you happen to be querying.



In-sample versus 80:20 holdout. The ranking of systems is preserved and the absolute gap is small for the deployed pipeline.

> **Result: the pipeline generalises.** Tuned on 8 JDs and applied blind to 2, the full pipeline scores **0.858** versus **0.887** in-sample — a gap of -0.029. The headline number was therefore not substantially inflated by tuning on the test queries. The honest caveat: with only 2 test JDs per fold the fold means are noisy (±0.12), so this rules out a *large* optimism bias, not a small one.

7.4.4 Three things the holdout revealed that the main table hides

1. The fusion weight selection is unstable. The weights chosen on each fold's training JDs were `(0, 0, 1), (0, 1, 2), (0, 1, 2), (0, 1, 2), (1, 0, 2)` as (keyword, AE, text). 3 different weightings were selected across the 5 folds; the modal choice (0.0, 1.0, 2.0) won 3 of 5. With 8 queries per fold the grid search is substantially fitting noise — exactly what §10 warned about — and the holdout turns that warning into a measurement rather than a hedge. It also means the specific weights in §7 are not a stable recommendation; the *family filter* is, because it helps in every fold regardless of the weights chosen.

2. The keyword AE is the least stable component. Its holdout Recall@10 is 0.511 with a standard deviation of 0.163 across folds, and it actually *improves* out-of-sample (+0.195). That is not a win — it means its score is dominated by which JDs happen to be in the fold, not by anything it learned. A component whose performance moves with the query split rather than with the task is not contributing reliably, and this reinforces the conclusion of §5.6 from a different direction.

3. The reranker's out-of-sample gain is negative. Its holdout Recall@10 (0.858) is *below* the fusion-plus-family system without it (0.880). In-sample the reranker looked like an improvement. The honest reading: its recall contribution is within noise, and its real value is the **ordering** (NDCG) and the **rationale text** it produces for each candidate — not the recall. We keep it for those reasons and now say so explicitly rather than claiming a recall gain.

One caveat on this specific comparison: the rerank scores were computed once, in step 07c, over each JD's in-sample top-20 pool. For a held-out fold the pool changes, so some pairs had no stored score and fell back to the neutral value 50. That substitution makes the out-of-sample rerank number a *lower* bound rather than an exact measurement, and it is the reason this section reports the reranker's value as NDCG and rationale quality rather than as a recall improvement.

> **What this does and does not prove.** It proves the headline number is not an artefact of tuning on the test queries. It does **not** make the evaluation fully out-of-sample, for two reasons that cannot be fixed with 10 JDs: (i) the ground-truth labels for the held-out JDs were built with the same protocol and the same model, so the *labels* are still in-distribution even when the query is held out; (ii) the AE is retrained per fold but the CV pool is shared, so the encoder still sees every CV. A genuinely clean evaluation needs new JDs with fresh human labels, which is improvement #8 in §9.

7.5 Top-10 shortlists with rationales

Each block is one JD. Grade is the ground-truth relevance; ✓ marks a relevant candidate. The right-hand column is the reranker's own one-line reason, i.e. exactly what a recruiter would see.

▶ **P01** · Regional Finance Controller · 4/4 relevant in top 10 Recall@10 = 1.000 · ceiling 1.00

▶ **P02** · Sales Manager in Indonesia · 2/2 relevant in top 10 Recall@10 = 1.000 · ceiling 1.00

▶ **P03** · GTM Marketing Manager - Lead · 6/6 relevant in top 10 Recall@10 = 1.000 · ceiling 1.00

▶ **P04** · Data Engineer · 5/5 relevant in top 10 Recall@10 = 1.000 · ceiling 1.00

▶ **P05** · Product Manager - WMS & Warehouse Integrations · 1/1 relevant in top 10 Recall@10 = 1.000 · ceiling 1.00

▶ **P06** · Deployment Strategist · 10/15 relevant in top 10 Recall@10 = 0.667 · ceiling 0.67

▶ **P07** · Marketing Manager (HK & SG) · 3/4 relevant in top 10 Recall@10 = 0.750 · ceiling 1.00

▶ **P08** · Field Service Engineer (FSE) · 1/1 relevant in top 10 Recall@10 = 1.000 · ceiling 1.00

▶ **P09** · Tech Lead · 10/22 relevant in top 10 Recall@10 = 0.455 · ceiling 0.45

▶ **P10** · Regional Lead Designer UI - UX · 2/2 relevant in top 10 Recall@10 = 1.000 · ceiling 1.00

8 · Error analysis

8.1 Per-JD breakdown

JD	title	|relevant|	retrieved	Recall@10	relevant CVs missed
P01	Regional Finance Controller	4	4	1.000	
P02	Sales Manager in Indonesia	2	2	1.000	
P03	GTM Marketing Manager - Lead	6	6	1.000	
P04	Data Engineer	5	5	1.000	
P05	Product Manager - WMS & Warehous	1	1	1.000	
P06	Deployment Strategist	15	10	0.667	CV-032, CV-035, CV-039, CV-043, CV-045
P07	Marketing Manager (HK & SG)	4	3	0.750	CV-074
P08	Field Service Engineer (FSE)	1	1	1.000	
P09	Tech Lead	22	10	0.455	CV-001, CV-003, CV-005, CV-009, CV-012
P10	Regional Lead Designer UI - UX	2	2	1.000	

8.2 Six failure modes visible in the data

F1 · The keyword bottleneck is the dominant error source

This is the clearest result in the project. `keyword` (merged skills) reaches 0.506 while `tfidf` on the same raw text reaches 0.717 — a +0.211 gap produced purely by throwing text away. The mean CV yields 26.7 extracted hard skills, but the pruned vocabulary used by the AE is only 12 canonical skills per document. A candidate whose relevant experience is described in prose ("led the migration of the billing pipeline") rather than listed as a keyword becomes invisible. Every downstream component inherits this loss, which is why the keyword AE sits at 0.316 and the text AE at 0.763.

F2 · Must-have weighting barely helps

`keyword_mh` (0.506) is statistically indistinguishable from `keyword` (0.506). The intended fix for "matching five nice-to-haves scores the same as matching five must-haves" does not work as expected, and the reason is diagnostic: after canonicalisation the JD's must-have list and its nice-to-have list overlap heavily, so most requirements end up at the same weight. The must/nice distinction is real in the JD text but is

destroyed by the extraction-then-merge pipeline. Fixing it needs the distinction to survive into the feature space (see §9, item 1).

F3 · Metric saturation hides real errors on large relevant sets

Seven of ten JDs have ≤10 relevant CVs, so Recall@10 = 1.0 is achievable and is achieved. That makes the headline number look near-perfect. The two JDs with large relevant sets — P06 (15 relevant) and P09 (22) — are where the pipeline actually loses candidates, and their ceiling is only 10/15 = 0.67 and 10/22 = 0.45. Any comparison that ignores the ceiling will over-credit the system on the easy JDs and under-credit it on the hard ones.

F4 · Seniority is not enforced by the retriever

Seniority enters the AE as one ordinal dimension out of 447 inputs, and the bottleneck is dominated by the skill block. Measured consequence: adding an explicit seniority/domain rerank (`hybrid_rerank`) changes Recall@10 by +0.002 — small, because the family gate has already removed most of the level mismatches. The remaining problem is inside a family: a mid-level engineer with an exactly matching skill list can outrank a lead with adjacent skills. The reranker partially fixes this because it reads the JD's stated level, but it is not a structural constraint.

F5 · Thin CVs are systematically under-ranked

CVs under ~1,600 characters yield few extracted skills, so their vectors are sparse and their cosine scores unstable. This is a representation problem, not a ranking problem: a candidate with genuine but under-described experience is penalised for brevity. It is visible in P09, where the missed candidates are disproportionately short CVs.

F6 · The reranker can be confidently wrong

On a handful of pairs the LLM reranker gives a low score to a candidate the ground truth calls relevant, because it only sees a 2,000-character excerpt and anchors on the must-have list. The retrieval prior in the blend limits the damage, but does not remove it. This is why `llm_rerank_only` (0.874) is slightly worse than the blend (0.887) despite having a higher Precision@10 on some JDs.

8.3 What we expected and did not find

Three hypotheses were tested and rejected. They are reported because negative results are what stop a report from becoming a list of successes:

- **"A bigger latent space will help."** Latent 64 scored worse than latent 32, and latent 16 worse than both. The problem was never capacity.
- **"More training will help."** 3,000 epochs scored worse than 1,800. The model overfits 110 documents quickly.

- **"More input blocks will help."** Adding domain and soft-skill one-hots dropped purity from 0.30 to 0.14, because those blocks are near-constant across documents and dominate the input norm.

9 · Improvement suggestions

#	change	expected effect	evidence from this run	cost
1	**Keep must-have and nice-to-have as separate feature blocks.** Two skill vectors per document, with separate overlap scores, instead of one merged set.	Attacks F2. The distinction is currently destroyed by merging.	`keyword_mh` ≈ `keyword` — the weighting had nothing to weight.	low
2	**Make the text embedding the primary representation and keywords the explanation layer.** The keywords stay in the product as the human-readable rationale, but retrieval runs on the embedding.	Attacks F1, the largest single error source.	`ae_text` 0.763 vs `ae_latent` 0.316 — same model, different input.	low
3	**Supervised re-ranker instead of an unsupervised AE.** With the labelled pairs now available, train a gradient-boosted ranker on the extracted features (must-have coverage, seniority gap, domain match, years gap).	Attacks F4 and F6 together, and uses labels the business already has.	The AE is unsupervised and optimises reconstruction, which §5.4 shows is the wrong objective.	medium
4	**Contrastive / triplet fine-tuning of the encoder.** Replace the reconstruction loss with InfoNCE over (JD, relevant CV, hard-negative CV) triples.	Trains the space to be *discriminative* rather than reconstructive.	Cos vs MSE reconstruction was already worth +0.16 purity; a ranking-aware loss should add more.	medium
5	**Two-tower architecture** with separate CV and JD encoders.	Removes the symmetric-input assumption of §5.1, so the model can express "this CV over-qualifies for this JD".	F4 is a symptom of the shared-input constraint.	medium
6	**Query expansion from the must-have clusters.** Retrieve with each must-have expanded into its synonym cluster, already computed in §4.	Attacks F5: thin CVs benefit most.	The merge clusters are already built and cost nothing to reuse.	low
7	**Chunk-level matching instead of document-level.** Score the JD against	Attacks F1 and F5 at once: prose experience becomes	Document-level cosine averages away the one	medium

#	change	expected effect	evidence from this run	cost
	each work-experience bullet and take the best-k.	matchable and short CVs stop being penalised for brevity.	relevant role in an otherwise irrelevant CV.	
8	**Human-audited labels at scale.** Have a recruiter adjudicate the 216 disagreements between the two annotators.	Removes label noise from every number in §7.	Binary κ is only 0.47.	high

10 · Assumptions and limitations

1. **The hired CV is not in the pool, so the task is relevance retrieval, not "find the hired person".** Verified directly (§2). Any evaluation that assumed otherwise would report recall 0 for every system.
2. **The ground truth is LLM-generated and its boundary is uncertain by about one grade.** This is the main threat to validity. Two annotators agree on the profession almost always (92% within ±1 grade) but only moderately on the relevant/not cut (binary κ = 0.47). Differences below ~0.05 Recall@10 between systems should be read as noise.
3. **The family gate defines the task.** Restricting evaluation to same-family pairs is a modelling choice, not a fact about the data: it encodes the judgement that a marketing manager is not a candidate for a data engineering role. It is defensible and it is stated, but a different granularity of "family" would produce different absolute numbers.
4. **The autoencoder is trained on the evaluation corpus.** It is unsupervised, so this is transductive learning, not label leakage — but the encoder has seen the CVs it ranks. In production it would be fit once on a large unlabelled corpus and frozen. §7.4 addresses the part that can be addressed: the AE is retrained per fold without the held-out JDs. It cannot address the shared CV pool, which is stated there.
5. **Fusion weights were selected on the same 10 JDs they are evaluated on.** Any hyper-parameter chosen on 10 queries is optimistically biased. The full grid is reported (§7.3) so the sensitivity is visible, the untuned equal-weight fusion is reported alongside, and **§7.4 quantifies the bias with a 5-fold 80:20 holdout**: the deployed pipeline scores 0.858 out-of-sample versus 0.887 in-sample, a gap of -0.029.
6. **Only 10 JDs.** Every mean in §7 averages over 10 queries; one JD moving from 0.2 to 0.6 shifts the mean by 0.04.
7. **No salary, notice-period, location or language filtering.** These are first-class recruiter filters in the project brief. They are extracted but deliberately not applied, so that the reported recall measures *semantic* matching only.
8. **Privacy.** All documents are the anonymised release. No name, email, phone or URL appears in the extracted features, and the LLM is local, so no document leaves the machine.

11 · Reproduction

```
# prerequisites: ollama with qwen3:8b and nomic-embed-text
python scripts/01_extract.py             # PDF → text JSON
python scripts/00_cv_family.py           # job-family map
python scripts/02_extract_keywords.py    # qwen3:8b keyword extraction (110 docs)
python scripts/03_ground_truth.py        # deterministic gate + LLM grading (1,000 pairs)
python scripts/03b_second_annotator.py   # independent re-annotation for κ
python scripts/04_verify.py              # dataset checks + statistics
python scripts/05_merge_keywords.py      # lexical + LLM + embedding merge
python scripts/06_autoencoder.py         # the brief's AE (keywords)
python scripts/06e1_ae_kw_txt.py          # AE variant: keywords + text
python scripts/06e2_ae_text.py            # AE variant: text only
python scripts/07_retrieve_eval.py       # scorers, fusion, metrics
python scripts/07b_family_classifier.py  # independent family classifier
python scripts/07c_llm_rerank.py         # LLM reranking of the top 20
python scripts/07d_apply_rerank.py       # fold rerank into the metrics
python scripts/13_holdout_eval.py        # 5-fold 80:20 out-of-sample check
python scripts/08_figures.py             # figures
python scripts/09_agreement.py           # inter-annotator agreement
python scripts/10_build_report.py        # this HTML report
python scripts/11_html_to_pdf.py         # PDF export
python scripts/12_transcript.py          # prompt transcript
```

Requirements: `pymupdf`, `torch`, `scikit-learn`, `numpy`, `matplotlib`, ollama with `qwen3:8b` and `nomic-embed-text`. Deterministic seeds throughout; the whole pipeline re-runs with no network access.

11.1 The diagnostic scripts

These are not part of the main pipeline — they are the experiments that produced the negative results in §5.4 and §8.3. They are included because a report that claims "a bigger latent space did not help" should ship the script that showed it.

script	question it answers	result
`06b_ae_sweep.py`	Is the AE better than doing nothing?	No — input space 0.36 vs AE 0.14–0.16
`06c1_ae_objective.py`	Does a direction-aware loss fix it?	Yes — 0.14 → 0.30 with cosine reconstruction
`06d1_ae_tune.py`	Does capacity, depth or training length help?	No — all variants ≤ 0.30
`06f1_ae_blocks.py`	Do domain / soft-skill blocks help?	No — 0.30 → 0.14
`06f2_ae_inputs.py`	Is the keyword representation or the AE the limit?	The input — text embedding reaches 0.34–0.38

script	question it answers	result
`06g1_ae_weights.py`	Can loss re-weighting recover the text block?	Only to 0.28, still below keyword-only
`04_verify.py`	Are the hired CVs in the pool?	No — zero overlap, which defines the task
`dump_profiles.py`	What are the 100 CVs about?	Generates `data/profiles.txt`, used to assign job families by hand
`99_check_refs.py`	Do the paths in this report exist?	Guard against a rename breaking the reproduction instructions

The `06b` – `06g` series reproduces the autoencoder negative results in §5.4 and the rejected hypotheses in §8.3.

STAT6207 Applied Deep Learning · Assignment 4 · local-only pipeline (ollama `qwen3:8b` + `nomic-embed-text`, PyTorch autoencoder) · 110 logged extraction calls · 4146 logged adjudication calls · report generated from the run artefacts.